

## **Project Title**

**Design and Implementation of a Text-to-SQL System Using Large Language Models and the Model Context Protocol**

**Student Name :** Ankita Sarkar

**Course :** B.Tech. in Computer Science and Engineering

**Institute name :** Kalinga Institute of Industrial Technology

**Project Guide / Mentor Name:** Diptendu Dutta

Period of Internship: 18<sup>th</sup> May 2026 – 31<sup>st</sup> July 2026

Report submitted to: IDEAS – Institute of Data  
Engineering, Analytics and Science Foundation, ISI  
Kolkata

# 1. Abstract

This project presents the development and evaluation of a **Text-to-SQL query generation system** using **Large Language Models (LLMs)** and the **Model Context Protocol (MCP)**. The objective is to translate natural language questions into valid DuckDB SQL queries, enabling users to retrieve database information without writing SQL manually. The system employs **Ollama** to run LLMs locally, with **DeepSeek-Coder 6.7B** used for SQL generation and **Llama 3.2** supported for model comparison. Two execution pipelines were implemented: a **Vanilla pipeline**, where SQL queries are executed directly on DuckDB, and an **MCP-based pipeline**, where SQL execution is performed through an MCP server. To improve the reliability of generated queries, schema-aware prompt engineering and SQL validation techniques were incorporated. The performance of the proposed system was evaluated using the **WikiSQL benchmark dataset** by comparing the generated SQL queries with the corresponding ground-truth queries. Experimental results demonstrate the effectiveness of integrating LLMs with DuckDB for natural language database querying while also highlighting the current limitations of general-purpose language models in Text-to-SQL tasks. The project provides a modular and extensible framework that can be enhanced with more advanced Text-to-SQL models and improved prompt engineering techniques. Overall, the work demonstrates the practical application of LLMs and MCP in building an intelligent and user-friendly database querying system.

## 2. Introduction

The rapid advancement of Artificial Intelligence (AI) and Natural Language Processing (NLP) has transformed the way users interact with computer systems. One of the most significant applications of these technologies is Text-to-SQL, which enables users to retrieve information from relational databases by asking questions in natural language instead of writing SQL queries manually. This approach improves accessibility for users who have little or no knowledge of SQL and simplifies database interaction.

This project, titled "Development and Evaluation of a Text-to-SQL Query Generation System Using Large Language Models and the Model Context Protocol (MCP)," focuses on building an intelligent system capable of converting natural language questions into valid SQL queries. The generated SQL queries are executed on a DuckDB database to retrieve the required information accurately. The project also investigates the use of the Model Context Protocol (MCP) as an alternative execution framework for database queries and compares it with the conventional execution approach.

The relevance of this project lies in its ability to bridge the gap between human language and database systems. As organizations increasingly rely on data-driven decision-making, there is a growing need for systems that allow non-technical users to access database information without requiring expertise in SQL. Text-to-SQL systems provide an efficient and user-friendly solution by automating the process of SQL query generation, thereby reducing human effort and improving productivity.

The project makes use of several modern technologies to achieve its objectives. Python is used as the primary programming language due to its simplicity and extensive ecosystem of

libraries. Ollama is employed as the local inference engine for running Large Language Models such as DeepSeek-Coder 6.7B and Llama 3.2. DuckDB serves as the relational database management system for executing SQL queries, while the Model Context Protocol (MCP) provides a standardized framework for database interaction. The WikiSQL benchmark dataset is used to evaluate the performance of the developed Text-to-SQL system.

## 2.1 Background Survey

Text-to-SQL has emerged as an important research area in Natural Language Processing because it enables users to interact with databases using conversational language. Early Text-to-SQL systems primarily relied on rule-based methods and traditional machine learning algorithms, which often struggled with complex database schemas and diverse natural language expressions. Recent advancements in **Large Language Models (LLMs)** have significantly improved the capability of automatically generating SQL queries from natural language inputs.

Several benchmark datasets have been introduced to evaluate Text-to-SQL systems, among which **WikiSQL** is one of the most widely used. It provides a collection of natural language questions, database schemas, and corresponding SQL queries for performance evaluation. In addition, tools such as **Ollama** allow developers to execute modern LLMs locally, providing better privacy, flexibility, and reduced dependence on cloud-based services. These developments have made it possible to build efficient and practical AI-assisted database querying systems.

## 2.2 Purpose of the Project

The primary purpose of this project is to develop an intelligent Text-to-SQL system capable of converting natural language questions into executable SQL queries. The project also aims to evaluate the effectiveness of Large Language Models for SQL generation and to compare a conventional SQL execution pipeline with an MCP-based execution pipeline. Furthermore, the project demonstrates the practical application of prompt engineering, DuckDB, and the WikiSQL benchmark dataset in developing an AI-powered database query system.

## 2.3 Technologies Used

| Technology          | Purpose                                                  |
|---------------------|----------------------------------------------------------|
| Python              | Primary programming language used for system development |
| Ollama              | Local inference engine for running Large Language Models |
| DeepSeek-Coder 6.7B | Primary model used for SQL query generation              |

|                              |                                                          |
|------------------------------|----------------------------------------------------------|
| Llama 3.2                    | Alternative Large Language Model used for comparison     |
| DuckDB                       | Relational database management system for SQL execution  |
| Model Context Protocol (MCP) | Standardized protocol for SQL query execution            |
| WikiSQL                      | Benchmark dataset for evaluating Text-to-SQL performance |
| Pandas                       | Data loading and preprocessing                           |
| Git & GitHub                 | Version control and project management                   |

### 3. Project Objective

- **To develop a Text-to-SQL query generation system** that converts natural language questions into valid DuckDB SQL queries using Large Language Models (LLMs).
- **To implement and compare two SQL execution pipelines**, namely the Vanilla pipeline and the Model Context Protocol (MCP)-based pipeline, for executing generated SQL queries.
- **To integrate Large Language Models through Ollama** for local SQL query generation using models such as DeepSeek-Coder 6.7B and Llama 3.2.
- **To evaluate the performance of the proposed Text-to-SQL system** using the WikiSQL benchmark dataset by comparing generated SQL queries with the corresponding ground-truth SQL queries.
- **To demonstrate the practical application of prompt engineering, DuckDB, and MCP** in developing an intelligent, modular, and user-friendly database querying system.

### 4. Methodology

**4.1 Overview :** The objective of this project was to develop a Text-to-SQL Query Generation System that converts natural language questions into executable SQL queries using Large Language Models (LLMs). The project also implements two execution pipelines: a Vanilla pipeline and a Model Context Protocol (MCP)-based pipeline. The complete system was developed using Python and evaluated using the WikiSQL benchmark dataset. The methodology adopted for this project consisted of data preparation, schema extraction, prompt engineering, SQL generation, SQL validation, query execution, and performance evaluation.

## 4.2 Development Methodology

### Step 1: Requirement Analysis

The project requirements were first analysed to understand the objectives of developing a Text-to-SQL system. A study was conducted on Text-to-SQL systems, Large Language Models (LLMs), DuckDB, the Model Context Protocol (MCP), and the WikiSQL benchmark dataset. The required software tools and Python libraries were identified before implementation.

### Step 2: Environment Setup

The development environment was configured with the required software and libraries.

The following tools were installed:

- Python
- Visual Studio Code
- Ollama
- DuckDB
- Pandas
- JSONLines
- MCP Python SDK
- Git
- GitHub

A virtual environment was created, and all project dependencies were installed using the `requirements.txt` file.

### Step 3: Database Preparation

A sample company database (`company.duckdb`) was used for testing the application. The database contained employee-related information required for executing SQL queries.

For evaluating the system, the WikiSQL benchmark dataset was used. The dataset consists of:

- Natural language questions
- Database table schemas
- Table contents
- Ground-truth SQL queries

Temporary DuckDB tables were created dynamically from the WikiSQL dataset during evaluation.

### Step 4: Schema Extraction

To ensure accurate SQL generation, the schema of the database was extracted automatically before generating SQL queries.

The schema extraction process involved:

- Reading all available tables
- Reading every column of each table
- Identifying column data types
- Formatting the schema into text

The extracted schema was included in the LLM prompt so that the generated SQL query used only valid tables and columns.

### **Step 5: Prompt Engineering**

Prompt engineering played a significant role in improving SQL generation.

The prompt provided the LLM with:

- Database schema
- Table names
- Column names
- User question
- SQL generation rules

The model was instructed to:

- Generate only one SQL query
- Avoid inventing tables
- Avoid inventing columns
- Return only SQL
- Use DuckDB syntax

This reduced hallucination and improved SQL correctness.

### **Step 6: SQL Query Generation**

Natural language questions were provided to the LLM through the Ollama Python API.

The project used:

- DeepSeek-Coder 6.7B
- Llama 3.2 (for comparison)

The model analysed the schema together with the user's question and generated a DuckDB-compatible SQL query.

### **Step 7: SQL Validation**

Before execution, every generated SQL query was validated.

Validation ensured:

- SQL syntax correctness
- Existing table names
- Existing column names

DuckDB's **EXPLAIN** command was used to detect invalid SQL statements before execution.

## **Step 8: SQL Execution**

Two execution approaches were implemented.

### **A. Vanilla Pipeline**

The generated SQL query was executed directly using the DuckDB Python API.

Workflow:

**User Question**

↓

**LLM**

↓

**Generated SQL**

↓

**DuckDB**

↓

**Result**

### **B. MCP Pipeline**

The generated SQL query was executed through an MCP server.

Workflow:

**User Question**

↓

**LLM**

↓

**Generated SQL**

↓

**MCP Client**

↓

**MCP Server**

↓

**DuckDB**

↓

**Result**

The MCP pipeline demonstrates how Model Context Protocol can standardize database interaction while keeping the LLM independent from the database implementation.

### **Step 9: WikiSQL Evaluation**

To evaluate the system objectively, the WikiSQL benchmark dataset was used.

The evaluation process involved:

- Loading 50 random test cases
- Creating temporary DuckDB tables
- Generating SQL using the LLM
- Converting WikiSQL logical forms into SQL
- Executing generated SQL
- Executing ground-truth SQL
- Comparing execution results
- Calculating overall accuracy

Failed test cases were stored separately for future analysis.

### **Step 10: Performance Evaluation**

The performance of the system was measured using execution-result matching.

Evaluation metrics included:

- Total test cases
- Correct predictions
- Incorrect predictions
- Accuracy percentage

Instead of comparing SQL strings directly, the outputs of generated SQL and ground-truth SQL were compared, providing a more reliable evaluation.



## 4.3 Data Collection

The project utilised two different sources of data.

### Company Database

A sample employee database stored in DuckDB was used for testing user queries during system development.

### WikiSQL Dataset

The WikiSQL benchmark dataset was used for performance evaluation. It contains natural language questions, relational tables, and corresponding SQL queries that serve as the ground truth.

## 4.4 Data Pre-processing

Before evaluation, the following preprocessing steps were performed:

- Loading JSONL files
- Reading table schemas
- Converting table rows into Pandas DataFrames
- Cleaning column names
- Registering temporary DuckDB tables
- Preparing schema descriptions for the LLM

These preprocessing steps ensured compatibility between the dataset and the SQL generation system.

## 4.5 Model Selection

DeepSeek-Coder 6.7B was selected as the primary model because it is specifically designed for code generation and demonstrated better SQL generation performance during experimentation.

## 4.6 Model Validation

The generated SQL queries were validated using two methods:

- SQL syntax validation using DuckDB's EXPLAIN command.
- Execution-result comparison against the ground-truth SQL query.

This ensured that evaluation was based on query correctness rather than textual similarity.

## 4.7 GitHub Repository

The complete project, including the source code, documentation, evaluation scripts, project report, and demonstration materials, has been maintained using Git and GitHub.

**GitHub Repository:**

<https://github.com/Ankita786-code/text2sql-with-mcp>

## 5. Data Analysis and Results

The developed Text-to-SQL system was evaluated using both a sample DuckDB database and the WikiSQL benchmark dataset. The evaluation focused on verifying the correctness of the SQL queries generated by the Large Language Model and comparing the execution results with the corresponding ground-truth SQL queries. Two execution pipelines, namely the Vanilla pipeline and the MCP-based pipeline, were tested to demonstrate the functionality of the proposed system.

### 5.1 Experimental Setup

| Parameter            | Value               |
|----------------------|---------------------|
| Programming Language | Python              |
| Database             | DuckDB              |
| LLM                  | DeepSeek-Coder 6.7B |
| Alternative Model    | Llama 3.2           |
| Dataset              | WikiSQL             |
| Evaluation Metric    | Execution Accuracy  |
| Total Test Cases     | 50                  |

### 5.2 Sample Results

Running the Vanilla Pipeline ([app\\_vanilla.py](#))

```
(venv) PS C:\Users\KIIT0001\Desktop\ISI project\text2sql-with-mcp> python app_vanilla.py
Ask your question: Total employees?

Generated SQL:

SELECT COUNT(*) FROM employee;

Result:

(20,)
```

### Running the MCP Pipeline (app\_mcp.py)

```
(venv) PS C:\Users\KIIT0001\Desktop\ISI project\text2sql-with-mcp> python app_mcp.py
Ask your question: Who has the lowest salary?

Generated SQL:

SELECT name FROM employee WHERE salary = (SELECT MIN(salary) FROM employee);

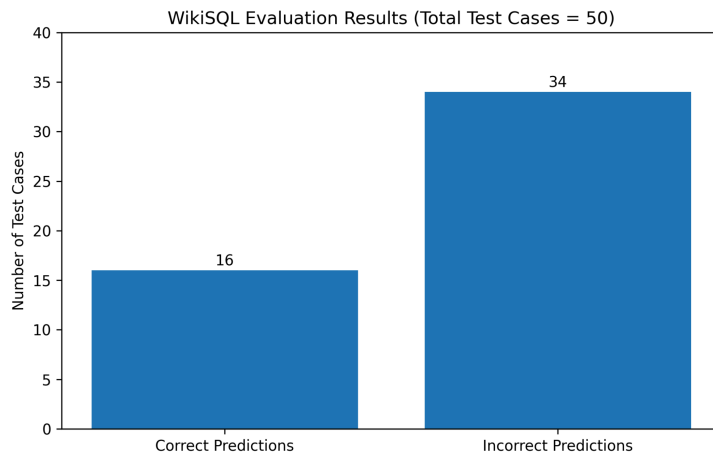
Executing through MCP...

Execution Error:
unhandled errors in a TaskGroup (1 sub-exception)
```

### WikiSQL Evaluation

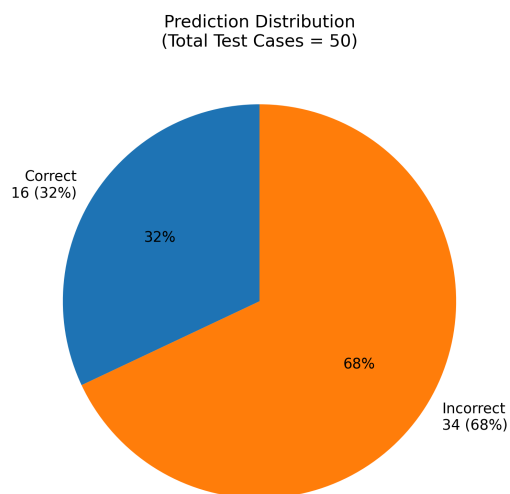
```
=====
Evaluation Summary
=====
Total Test Cases : 50
Matched          : 16
Not Matched      : 34
Accuracy         : 32.00%
```

## WikiSQL Evaluation Bar Chart



The WikiSQL evaluation bar chart presents the performance of the proposed Text-to-SQL system on 50 test cases. Out of the total test cases, 16 SQL queries were generated correctly, while 34 queries did not match the expected execution results, resulting in an overall execution accuracy of 32%.

## WikiSQL Evaluation Pie Chart



The WikiSQL evaluation pie chart illustrates the distribution of prediction outcomes obtained during system evaluation. The proposed Text-to-SQL system achieved 32% correct predictions and 68% incorrect predictions, indicating that while the system successfully handles many natural language queries, further improvements in model selection and prompt engineering are required to increase SQL generation accuracy.

## 6. Conclusion

The primary objective of this project was to develop a **Text-to-SQL Query Generation System** using **Large Language Models (LLMs)** and the **Model Context Protocol (MCP)**. The developed system successfully converts natural language questions into executable SQL queries using Ollama as the local inference engine. Two execution pipelines, namely the **Vanilla pipeline** and the **MCP-based pipeline**, were implemented and tested successfully.

The system was evaluated using the **WikiSQL benchmark dataset**, where **50 test cases** were executed. The experimental results achieved an **execution accuracy of 32%**, with **16 correctly generated SQL queries** and **34 incorrect predictions**. Although the system performed well for many simple SQL queries, the evaluation also highlighted the challenges of handling complex SQL generation tasks using general-purpose Large Language Models.

The project demonstrates that integrating Large Language Models with DuckDB enables users to retrieve database information using natural language instead of manually writing SQL queries. Furthermore, the implementation of the MCP pipeline shows how the Model Context Protocol can provide a standardized and modular approach for database interaction. Overall, the project successfully achieved its objectives and provides a strong foundation for future research in AI-assisted database querying systems.

## Future Work

The following improvements can be considered in future work:

- Integrate advanced Text-to-SQL models specifically designed for SQL generation.
- Improve prompt engineering techniques to increase SQL generation accuracy.
- Extend support for multiple relational database management systems such as MySQL and PostgreSQL.
- Support complex SQL queries involving joins, nested queries, and multiple tables.
- Develop a web-based graphical user interface to improve usability for non-technical users.

## 7. APPENDICES

You can list the references as follows:

1. Ollama. *Ollama Documentation*. <https://ollama.com>
2. DuckDB. *DuckDB Documentation*. <https://duckdb.org/docs>
3. WikiSQL Dataset. *Salesforce Research*. <https://github.com/salesforce/WikiSQL>
4. Model Context Protocol. *Official Documentation*. <https://modelcontextprotocol.io>
5. Python Software Foundation. *Python Documentation*. <https://docs.python.org/3>
6. Pandas Development Team. *Pandas Documentation*. <https://pandas.pydata.org/docs>
7. GitHub Documentation. <https://docs.github.com>

## **GITHUB REPOSITORY**

### **Source Code Repository**

<https://github.com/Ankita786-code/text2sql-with-mcp>